

## Lec 4

# Complexity Analysis, Time Complexity & Asymptotic Notation

Date: 20th July 2026

---

## 1. What is Complexity Analysis?

Complexity analysis is the study of how the resource usage of an algorithm — **time** (number of operations) and **space** (memory) — grows as the input size  $n$  grows. We don't measure this in actual seconds (that depends on hardware, compiler, language), we measure it in terms of **how the number of basic operations scales with  $n$** .

Two main types:

- **Time Complexity** — how the number of steps grows with  $n$ .
- **Space Complexity** — how the memory used grows with  $n$ .

This document focuses on **time complexity**.

## Why not just measure runtime in seconds?

Because runtime depends on:

- CPU speed, cache, architecture
- Programming language / compiler optimizations
- Background load on the machine

So instead, we count **basic operations** (comparisons, assignments, arithmetic operations) as a function of input size  $n$ , giving a hardware-independent measure.

---

## 2. Time Complexity

Time complexity  $T(n)$  is a function that gives the number of primitive operations executed by an algorithm for an input of size  $n$ .

For example, for a loop that runs  $n$  times, doing constant work each time:

$$T(n) = c_1 \cdot n + c_2$$

where  $c_1, c_2$  are constants (setup, loop overhead). As  $n \rightarrow \infty$ , the constants stop mattering — only the **growth rate** ( $n$  here) matters. That's exactly what asymptotic notation captures.

We generally care about three cases:

- **Best case** — minimum steps (e.g., searching finds the element immediately)
- **Average case** — expected steps over all inputs
- **Worst case** — maximum steps (this is what we usually analyze, since it gives a guarantee)

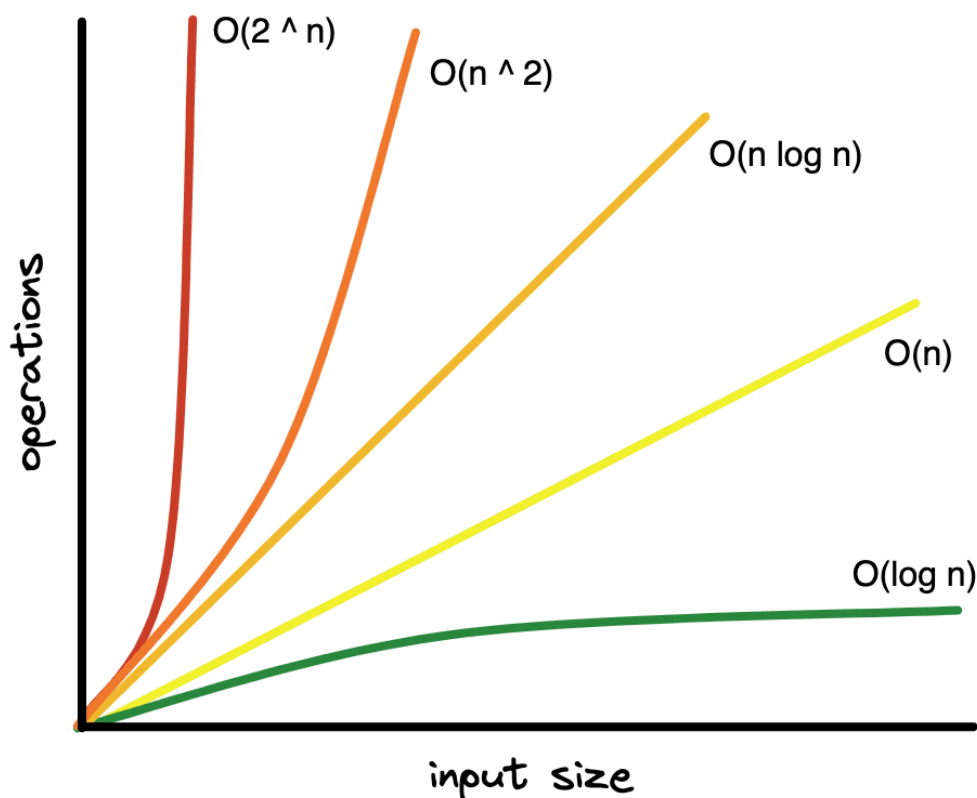
---

## 3. Asymptotic Notation

Asymptotic notation describes the **limiting behavior** of a function as  $n \rightarrow \infty$ , ignoring constant factors and lower-order terms. There are three key notations: **Big-O**, **Big-Omega** ( $\Omega$ ), and **Big-Theta** ( $\Theta$ ).

### 3.1 Big-O Notation — Upper Bound

#### Big O Complexity Chart



Big-O describes the **worst-case growth rate** — an upper bound on the function. It answers: *"the algorithm will never take more than this, asymptotically."*

**Formal definition:**

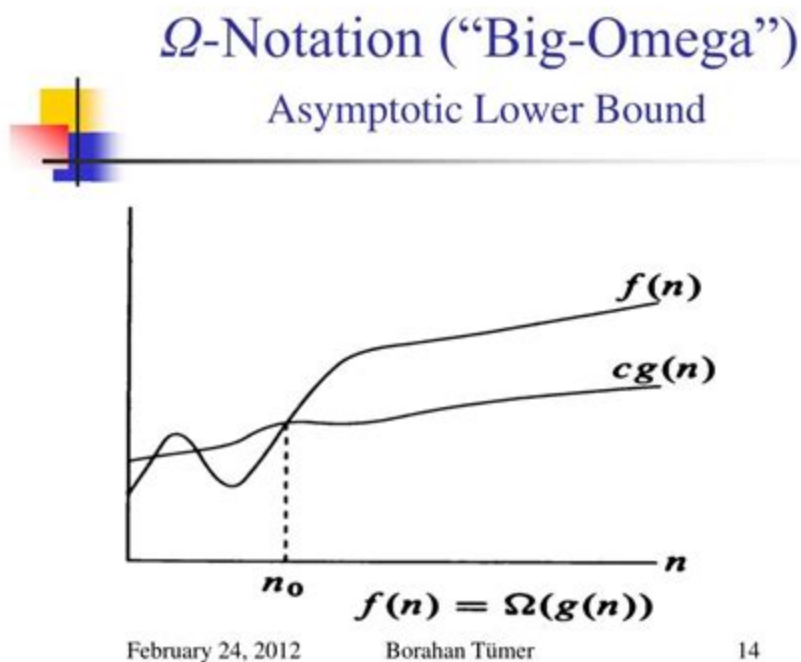
$$f(n) = O(g(n)) \iff \exists, c > 0, n_0 > 0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

In words:  $f(n)$  grows **no faster than**  $g(n)$ , up to a constant factor, once  $n$  is large enough.

**Example:** If  $f(n) = 3n^2 + 5n + 2$ , then  $f(n) = O(n^2)$ , because for  $n_0 = 1$  and  $c = 10$ :

$$3n^2 + 5n + 2 \leq 10n^2 \quad \text{for all } n \geq 1$$

## 3.2 Big-Omega Notation — Lower Bound



Big-Omega describes the **best-case growth rate** — a lower bound. It answers: *"the algorithm will always take at least this long, asymptotically."*

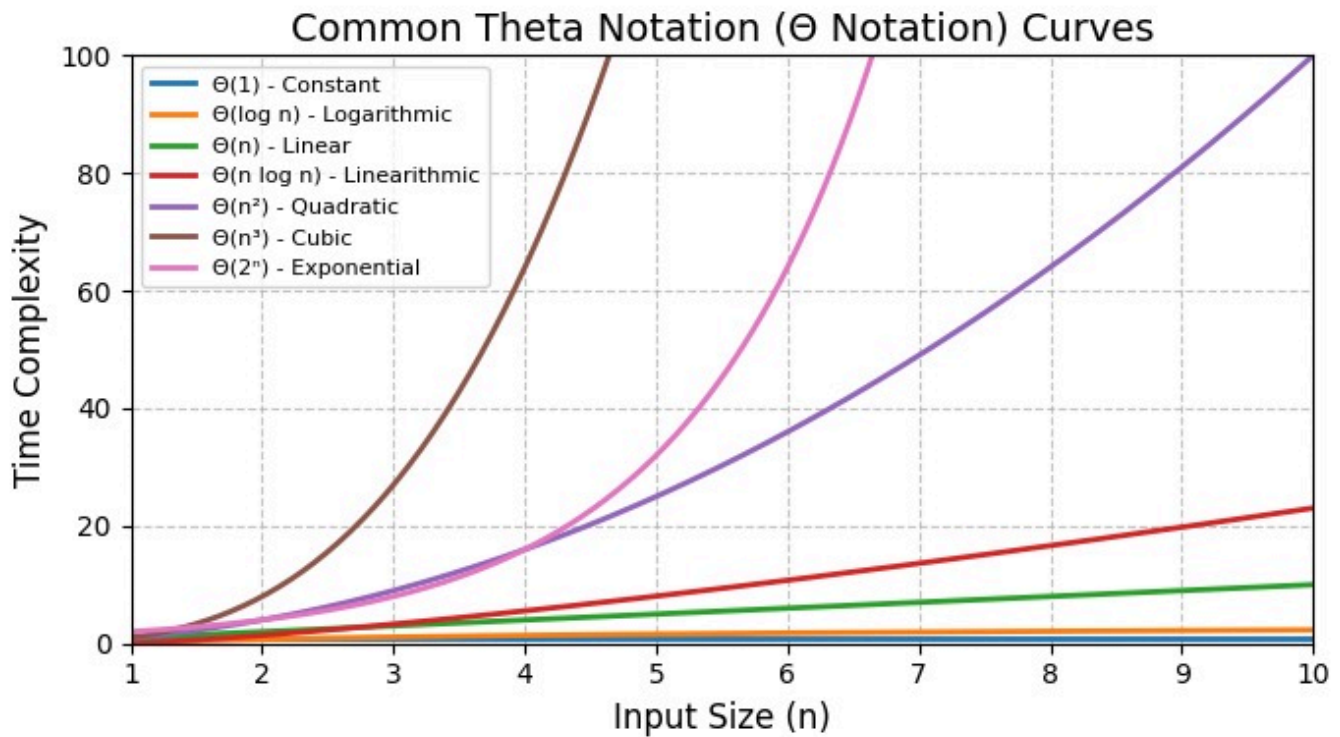
**Formal definition:**

$$f(n) = \Omega(g(n)) \iff \exists, c > 0, n_0 > 0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \quad \forall n \geq n_0$$

**Example:**  $f(n) = 3n^2 + 5n + 2$  is  $\Omega(n^2)$ , because for  $c = 3$  and  $n_0 = 1$ :

$$3n^2 \leq 3n^2 + 5n + 2 \quad \text{for all } n \geq 1$$

## 3.3 Big-Theta Notation — Tight Bound



Big-Theta means the function is **bounded both above and below** by  $g(n)$  — i.e.,  $g(n)$  is a *tight* estimate of  $f(n)$ 's growth.

**Formal definition:**

$$f(n) = \Theta(g(n)) \iff \exists, c_1, c_2 > 0, n_0 > 0 \text{ such that } c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$$

Equivalently:  $f(n) = \Theta(g(n))$  **if and only if**  $f(n) = O(g(n))$  **and**  $f(n) = \Omega(g(n))$ .

**Example:**  $f(n) = 3n^2 + 5n + 2 = \Theta(n^2)$ , since we showed both the  $O(n^2)$  and  $\Omega(n^2)$  bounds above with the same  $g(n) = n^2$ .

### 3.4 Intuition Summary

| Notation       | Bound Type                        | Real-world analogy                                                           |
|----------------|-----------------------------------|------------------------------------------------------------------------------|
| $O(g(n))$      | Upper bound ("at most")           | Worst case — "I'll finish by this time at the latest"                        |
| $\Omega(g(n))$ | Lower bound ("at least")          | Best case — "I can't possibly finish before this time"                       |
| $\Theta(g(n))$ | Tight bound ("exactly this rate") | Average/guaranteed — "I'll finish at <i>this</i> rate, no faster, no slower" |

A useful visual: if you plot  $f(n)$ , then  $O(g(n))$  is a curve that stays **on or below**  $c \cdot g(n)$  eventually;  $\Omega(g(n))$  stays **on or above**  $c \cdot g(n)$  eventually;  $\Theta(g(n))$  is **sandwiched between** two scaled copies of  $g(n)$ .

### 3.5 Common Growth Rates (fastest to slowest)

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

| Complexity    | Name         | Example algorithm                            |
|---------------|--------------|----------------------------------------------|
| $O(1)$        | Constant     | Array index access                           |
| $O(\log n)$   | Logarithmic  | Binary search                                |
| $O(n)$        | Linear       | Single loop / linear search                  |
| $O(n \log n)$ | Linearithmic | Merge sort, heap sort                        |
| $O(n^2)$      | Quadratic    | Nested loops / bubble sort                   |
| $O(n^3)$      | Cubic        | Triple nested loops / naive matrix mult.     |
| $O(2^n)$      | Exponential  | Naive recursive Fibonacci, subset generation |
| $O(n!)$       | Factorial    | Brute-force traveling salesman               |

## 4. Rules for Computing Complexity

1. **Drop constants:**  $O(5n) = O(n)$ .
2. **Drop lower-order terms:**  $O(n^2 + n) = O(n^2)$ .
3. **Sequential statements add:** two loops one after another  $\rightarrow$  complexities add:  
 $O(n) + O(n^2) = O(n^2)$  (keep the dominant term).
4. **Nested loops multiply:** a loop of  $O(n)$  inside a loop of  $O(n) \rightarrow O(n) \times O(n) = O(n^2)$ .
5. **Recursive calls** are analyzed using a **recurrence relation**, often solved via the **recursion tree method**, **substitution method**, or the **Master Theorem**.

## 5. Worked Examples (Increasing Difficulty)

### Example 1 — Single Loop (Linear Time)

```
def sum_array(arr):
    total = 0                # O(1)
    for x in arr:            # runs n times
        total += x           # O(1) each
    return total             # O(1)
```

#### Step-by-step:

- Initialization `total = 0`  $\rightarrow$  constant time,  $c_1$ .

- The loop runs once for every element in `arr`, so  $n$  iterations.
- Each iteration does one addition and one assignment  $\rightarrow$  constant time  $c_2$  per iteration.
- Return statement  $\rightarrow$  constant,  $c_3$ .

$$T(n) = c_1 + c_2 \cdot n + c_3$$

Dropping constants and lower-order terms:

$$T(n) = O(n)$$

This is the simplest non-trivial case: **one loop over the input  $\rightarrow$  linear time.**

## Example 2 — Nested Loop (Quadratic Time)

```
def has_duplicate_pair(arr):
    n = len(arr)
    for i in range(n):          # outer loop: n times
        for j in range(i+1, n): # inner loop: up to n times
            if arr[i] == arr[j]:
                return True
    return False
```

**Step-by-step:**

- Outer loop runs for  $i = 0, 1, \dots, n-1 \rightarrow n$  iterations.
- For each  $i$ , inner loop runs  $(n-i-1)$  times.
- Total number of comparisons:

$$T(n) = \sum_{i=0}^{n-1} (n-i-1) = (n-1) + (n-2) + \dots + 1 + 0 = \frac{n(n-1)}{2}$$

- Expanding:  $T(n) = \frac{n^2}{2} - \frac{n}{2}$ .
- Drop constants and lower-order term  $(-\frac{n}{2})$ :

$$T(n) = O(n^2)$$

**Key insight:** even though the inner loop shrinks each time, the *sum* of a shrinking series over  $n$  terms is still  $\Theta(n^2)$  (a triangular number), not  $O(n)$ .

## Example 3 — Loop with Multiplicative Step (Logarithmic Time)

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

### Step-by-step:

- Each iteration eliminates **half** of the remaining search space.
- After 1 iteration: search space is  $n/2$ .
- After 2 iterations:  $n/4 = n/2^2$ .
- After  $k$  iterations:  $n/2^k$ .
- The loop stops when the search space shrinks to 1:

$$\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n$$

$$T(n) = O(\log n)$$

**Key insight:** whenever the input is repeatedly **divided** by a constant factor each step, you get a **logarithmic** number of steps.

---

## Example 4 — Two Sequential Loops Followed by a Nested Loop (Combining Rules)

```
def process(arr):
    n = len(arr)
    total = 0
    for x in arr:                # Loop A: O(n)
        total += x

    max_val = arr[0]
    for x in arr:                # Loop B: O(n)
        if x > max_val:
            max_val = x
```

```

for i in range(n):          # Loop C (nested):  $O(n^2)$ 
    for j in range(n):
        print(arr[i], arr[j])

return total, max_val

```

### Step-by-step:

- Loop A: single pass  $\rightarrow O(n)$ .
- Loop B: single pass  $\rightarrow O(n)$ .
- Loop C: nested loop, both ranging over full  $n \rightarrow O(n^2)$ .
- Since these run **sequentially** (one after another, not nested in each other), we **add** their complexities:

$$T(n) = O(n) + O(n) + O(n^2) = O(2n + n^2)$$

- Drop constants and lower-order terms —  $n^2$  dominates  $2n$  as  $n \rightarrow \infty$ :

$$T(n) = O(n^2)$$

**Key insight:** when combining blocks of code that run one after the other, the **overall complexity is the maximum (dominant) term**, not the sum of orders.

## Example 5 — Simple Linear Recursion

```

def factorial(n):
    if n <= 1:          # base case:  $O(1)$ 
        return 1
    return n * factorial(n - 1)  # recursive case:  $O(1) + T(n-1)$ 

```

### Step-by-step — setting up the recurrence:

$$T(n) = T(n - 1) + O(1), \quad T(1) = O(1)$$

### Solving by substitution (unrolling):

$$T(n) = T(n - 1) + c = [T(n - 2) + c] + c = T(n - 2) + 2c = \dots = T(1) + (n - 1)c$$

Since  $T(1) = O(1)$ :

$$T(n) = O(1) + (n - 1) \cdot c = O(n)$$

**Key insight:** a recursive function that makes **one recursive call** reducing  $n$  by a constant amount, doing constant work otherwise, is  $O(n)$  — equivalent to a single loop.



## Example 6 — Divide and Conquer Recursion (Merge Sort)

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])      # T(n/2)
    right = merge_sort(arr[mid:])     # T(n/2)
    return merge(left, right)        # O(n) to merge
```

**Step-by-step — the recurrence:**

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n), \quad T(1) = O(1)$$

This matches the **Master Theorem** form:

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d)$$

with  $a = 2$  (two subproblems),  $b = 2$  (each half the size),  $d = 1$  (merging is linear).

**Master Theorem statement:**

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

**Applying it:**  $\log_b a = \log_2 2 = 1 = d$ . This is the **middle case**, so:

$$T(n) = O(n^d \log n) = O(n \log n)$$

**Verifying by recursion tree (alternative method):**

- At depth 0: 1 problem of size  $n$ , cost  $n$ .
- At depth 1: 2 problems of size  $n/2$ , cost  $2 \cdot \frac{n}{2} = n$ .
- At depth 2: 4 problems of size  $n/4$ , cost  $4 \cdot \frac{n}{4} = n$ .
- Every level costs  $O(n)$  in total.
- Number of levels until subproblem size reaches 1:  $\log_2 n$ .
- Total cost:

$$T(n) = n \cdot \log_2 n = O(n \log n)$$

Both methods agree:  $T(n) = O(n \log n)$ .

**Key insight:** dividing the problem into multiple subproblems (here, 2) each of reduced size, plus linear combination work, gives **linearithmic** time — much better than the  $O(n^2)$  of

## Example 7 — Naive Exponential Recursion (Fibonacci)

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

**Step-by-step — the recurrence:**

$$T(n) = T(n - 1) + T(n - 2) + O(1), \quad T(0) = T(1) = O(1)$$

This is **not** divide-and-conquer (subproblems overlap and don't shrink by a constant factor) — the Master Theorem does not directly apply. Instead, use the **recursion tree method**:

- Each call to `fib(n)` spawns 2 further calls, until reaching a base case.
- The recursion tree has branching factor 2, and depth roughly  $n$  (since each level reduces  $n$  by only 1 or 2).
- Number of nodes in a binary tree of depth  $n$  is bounded by  $2^n$ .

**Formal bound (upper bound via substitution):** guess  $T(n) \leq c \cdot 2^n$  and verify:

$$T(n) = T(n - 1) + T(n - 2) + O(1) \leq c \cdot 2^{n-1} + c \cdot 2^{n-2} + O(1) \leq c \cdot 2^n$$

(since  $2^{n-1} + 2^{n-2} = \frac{3}{4} \cdot 2^n < 2^n$ , leaving room to absorb the  $O(1)$  term). This confirms:

$$T(n) = O(2^n)$$

A tighter analysis shows  $T(n) = \Theta(\phi^n)$  where  $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$  is the golden ratio, but  $O(2^n)$  is the standard loose bound taught at this stage.

**Key insight:** when a recursive function calls itself **more than once** without shrinking the problem size by a large enough factor (here, each call only reduces  $n$  by 1 or 2, but there are 2 calls per level), the number of calls **grows exponentially** — this is why naive recursive Fibonacci is impractical for large  $n$ , and why memoization / dynamic programming (reducing it to  $O(n)$ ) matters.

---

## 6. Summary Table of All Examples

| Example                      | Structure                         | Recurrence / Reasoning          | Complexity    |
|------------------------------|-----------------------------------|---------------------------------|---------------|
| 1. Sum of array              | Single loop                       | $T(n) = c \cdot n$              | $O(n)$        |
| 2. Duplicate pair check      | Nested loop (triangular)          | $T(n) = \frac{n(n-1)}{2}$       | $O(n^2)$      |
| 3. Binary search             | Loop halving search space         | $n/2^k = 1$                     | $O(\log n)$   |
| 4. Sequential + nested loops | Additive + multiplicative rule    | $O(n) + O(n) + O(n^2)$          | $O(n^2)$      |
| 5. Factorial                 | Linear recursion                  | $T(n) = T(n-1) + O(1)$          | $O(n)$        |
| 6. Merge sort                | Divide and conquer                | $T(n) = 2T(n/2) + O(n)$         | $O(n \log n)$ |
| 7. Naive Fibonacci           | Overlapping exponential recursion | $T(n) = T(n-1) + T(n-2) + O(1)$ | $O(2^n)$      |

## 7. Key Takeaways

- Time complexity measures **growth rate**, not actual runtime.
- $O$ ,  $\Omega$ ,  $\Theta$  describe **upper**, **lower**, and **tight** bounds respectively.
- Loops: single loop  $\rightarrow$  linear; nested loops  $\rightarrow$  multiply; sequential blocks  $\rightarrow$  add (keep dominant term).
- Halving the problem each step  $\rightarrow$  logarithmic.
- Recursive complexity is found by setting up and solving a **recurrence relation** — via substitution, recursion trees, or the Master Theorem (for divide-and-conquer forms  $T(n) = aT(n/b) + O(n^d)$ ).
- Multiple overlapping recursive calls without sufficient shrinkage  $\rightarrow$  exponential blow-up; this motivates techniques like memoization/dynamic programming.